

实验四报告

唐宇恩 251220100

2026 年 5 月 22 日

1 4 位先行进位加法器 CLA

1.1 整体方案设计

本实验以 4 个全加器 (FA) 和一个进位预生成/传播单元 (CLU) 组成 4 位先行进位加法器。端口定义：

端口符号	端口功能
输入端 $A_3 \sim A_0$	被加数 (高位到低位)
输入端 $B_3 \sim B_0$	加数 (高位到低位)
输入端 C_0	初始进位输入
输出端 $S_3 \sim S_0$	和输出 (高位到低位)
输出端 C_4	最高位进位输出

1.2 实验电路图

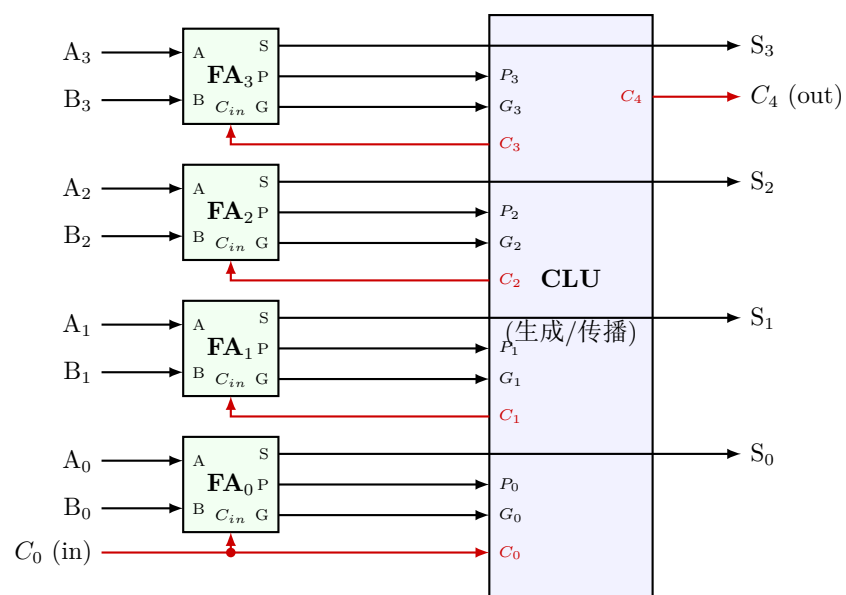


图 1: 4 位先行进位加法器原理图

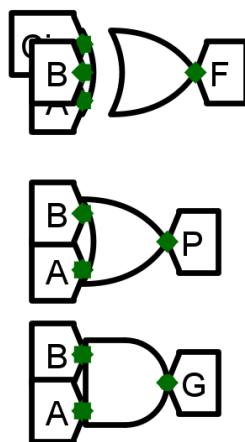


图 2: 全加器电路图

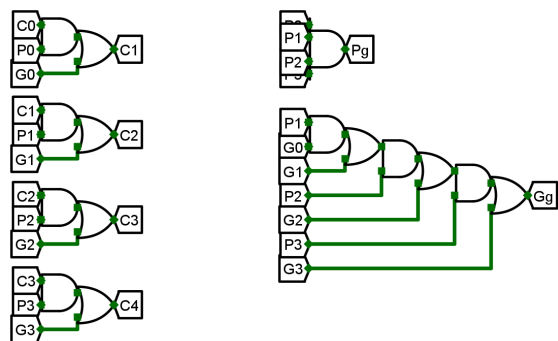


图 3: CLU 电路图

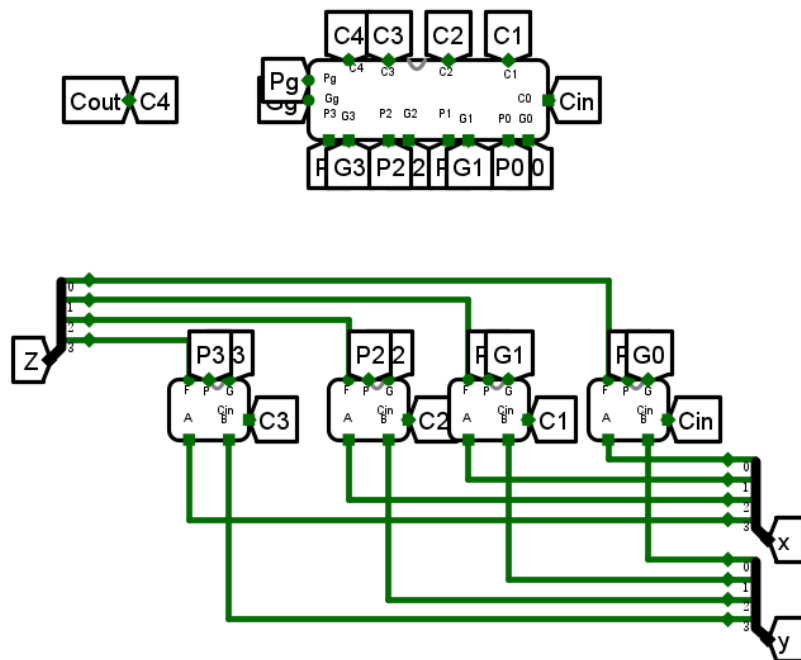


图 4: 4 位先行进位加法器电路图

1.3 仿真测试

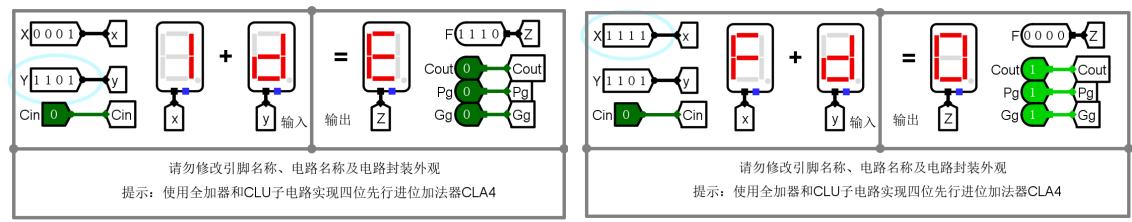


图 5: CLA 仿真测试

1.4 错误分析

本次实验没有遇到问题和错误.

2 16 位两级先行进位加法器

2.1 整体方案设计

本实验采用将 4 位/16 位先行进位加法器作为模块进行分级组合，实现 16 位先行进位加法器。端口定义：

端口符号	端口功能
输入端 $A_{15} \sim A_0$	被加数
输入端 $B_{15} \sim B_0$	加数
输入端 C_0	初始进位输入
输出端 $S_{15} \sim S_0$	和输出
输出端 C_{16}	最高位进位输出

2.2 实验电路图

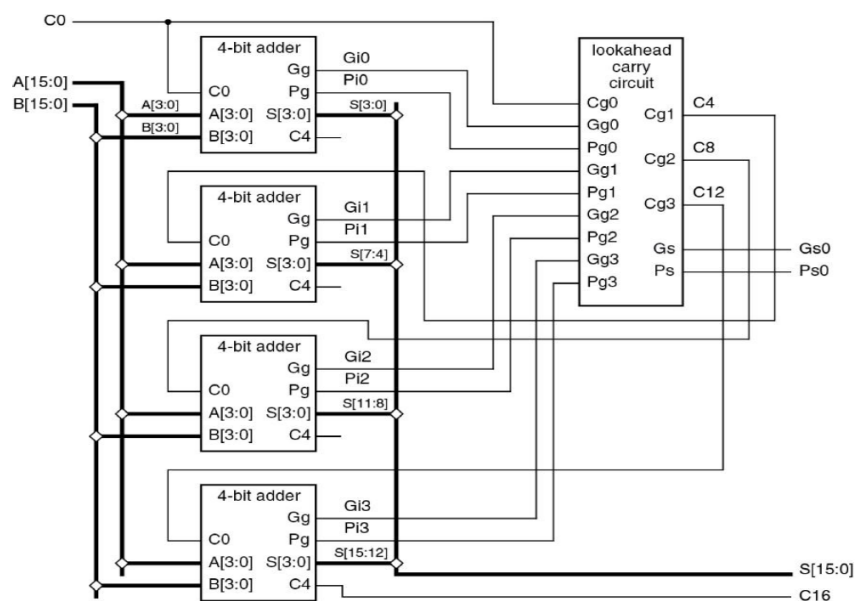


图 6: 16 位两级先行进位加法器原理图

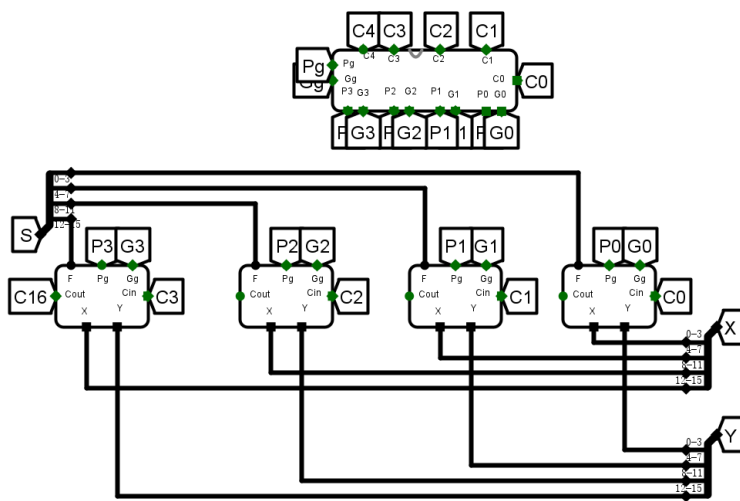


图 7: 16 位两级先行进位加法器电路图

2.3 仿真测试

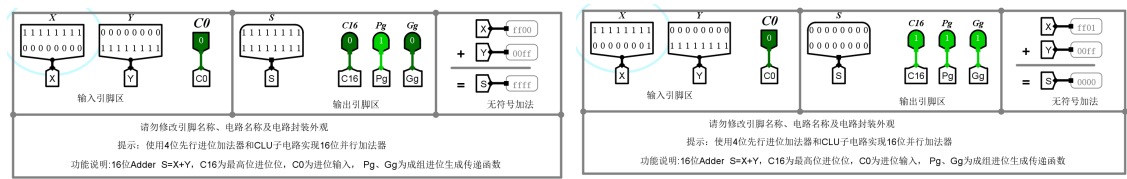


图 8: 16 位两级 CLA 仿真测试图

2.4 错误分析

本次实验没有遇到问题和错误.

3 32 位快速加法器

3.1 整体方案设计

本实验将两级 16 位先行进位加法器并联组合为 32 位快速加法器，同时生成必要的标志位 (溢出、零、符号等)。端口定义：

端口符号	端口功能
输入端 $A_{31} \sim A_0$	被加数
输入端 $B_{31} \sim B_0$	加数
输入端 C_0	初始进位输入
输出端 $S_{31} \sim S_0$	和输出
输出端 C_{32}	最高位进位输出
输出端 $Zero$	零标志
输出端 $Overflow$	溢出标志

3.2 实验电路图

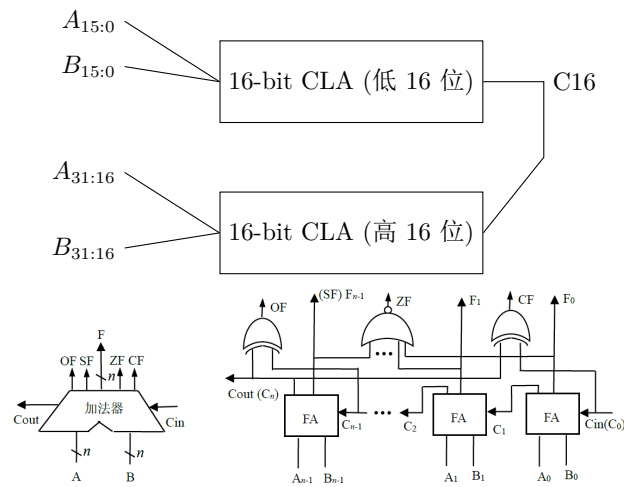


图 9: 32 位快速加法器原理图及标志位生成原理图

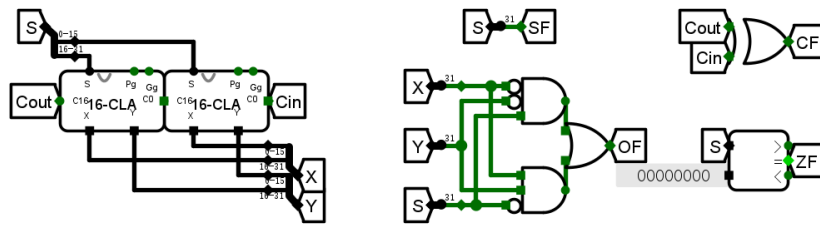


图 10: 32 位快速加法器电路图

3.3 仿真测试

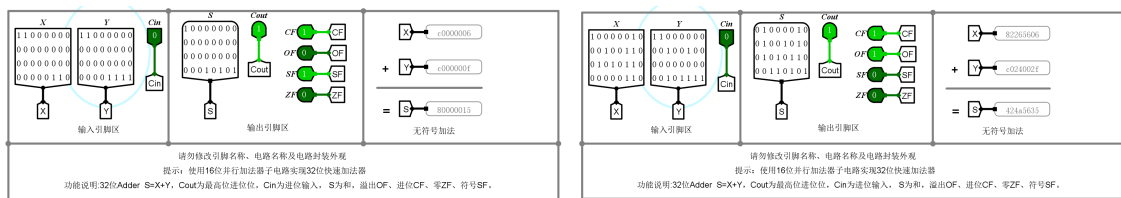


图 11: 32 位快速加法器仿真测试图

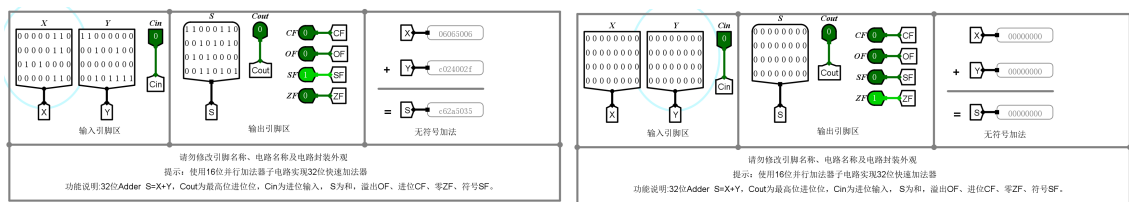


图 12: 32 位快速加法器仿真测试（测试 3/测试 4）

3.4 错误分析

本次实验没有遇到问题和错误.

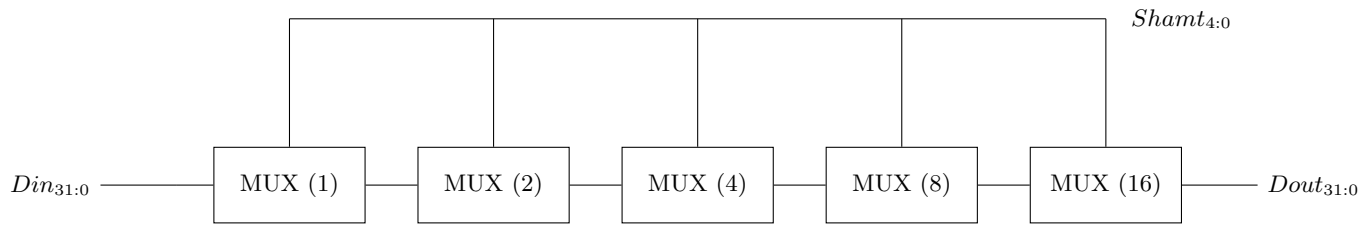
4 32 位桶形移位器

4.1 整体方案设计

本实验设计一个 32 位桶形 (阶梯) 移位器, 支持逻辑左/右移与算术右移等操作。使用 5 级多路选择器逐级选择移位量 (1,2,4,8,16), 实现任意 0 31 位的移位。端口定义:

端口符号	端口功能
输入端 $D_{31} \sim D_0$	32 位数据输入
输入端 $S_4 \sim S_0$	5 位移位控制 (二进制表示移位置)
输入端 $Mode$	移位模式 (00: 保持, 01: 逻辑左移, 10: 逻辑右移, 11: 算术右移)
输出端 $Y_{31} \sim Y_0$	32 位移位结果输出

4.2 实验电路图



各级 MUX 对应移位置: 1、2、4、8、16 bit

控制信号由 $Shamt_{4:0}$ 决定

图 13: 32 位桶形移位器原理图

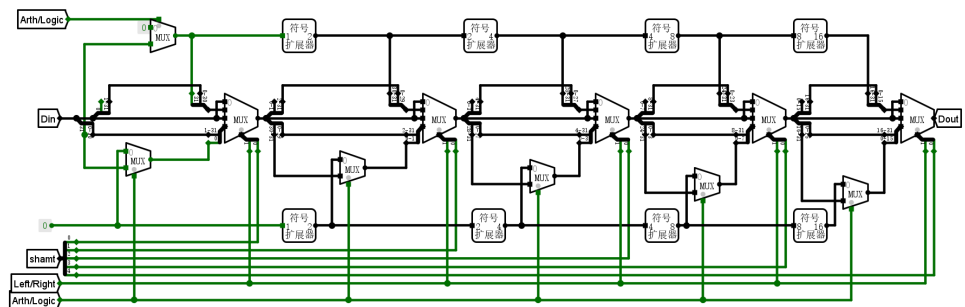


图 14: 32 位桶形移位器电路图

4.3 仿真测试

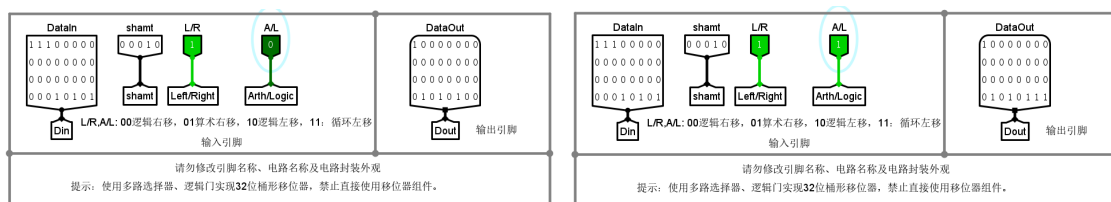


图 15: 桶形移位器仿真测试

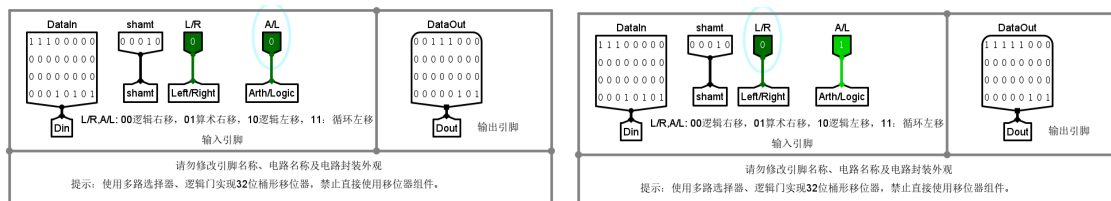


图 16: 桶形移位器仿真测试（测试 3/测试 4）

4.4 错误分析

本次实验没有遇到问题和错误.

5 RV32I 算术逻辑部件

5.1 整体方案设计

本实验实现 RV32I 指令集的算术逻辑部件 (ALU)，支持加、减、逻辑与/或/非、移位、比较等常见操作，并输出标志位。端口定义：

端口符号	端口功能
输入端 $A_{31:0}$	操作数 A
输入端 $B_{31:0}$	操作数 B
输入端 $ALUctr_{3:0}$	操作选择码
输出端 $Result_{31:0}$	结果
输出端 $Zero$	零标志
输出端 $Sign$	符号位
输出端 $Overflow$	溢出标志

5.2 实验电路图

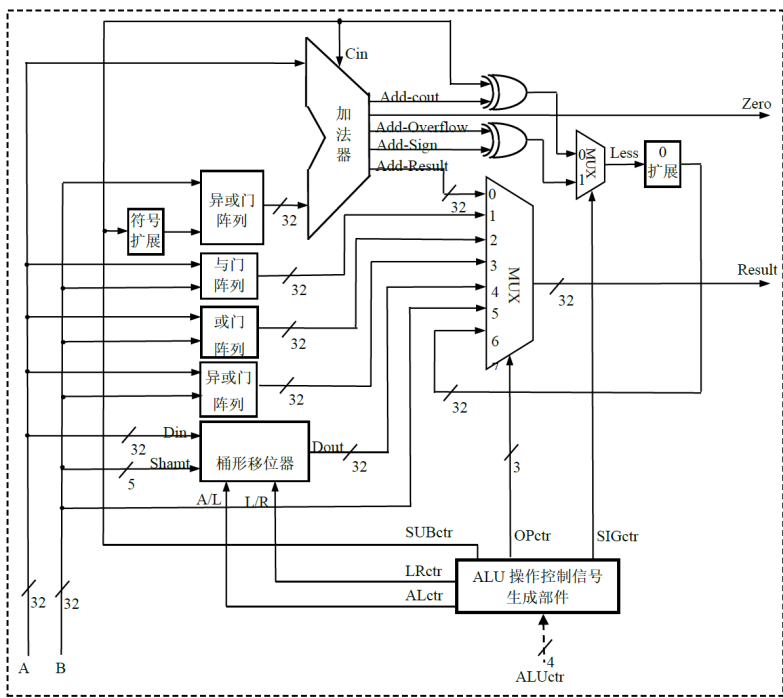


图 17: RV32I 算术逻辑部件原理图

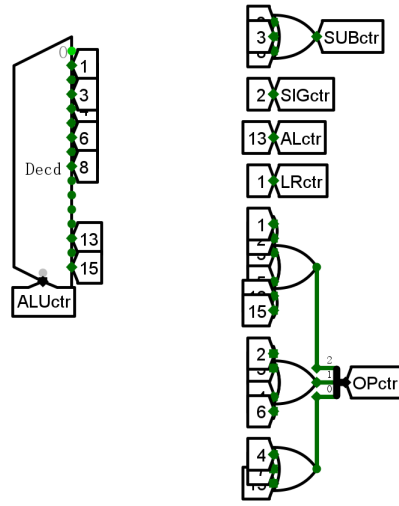


图 18: ALUctr 电路图

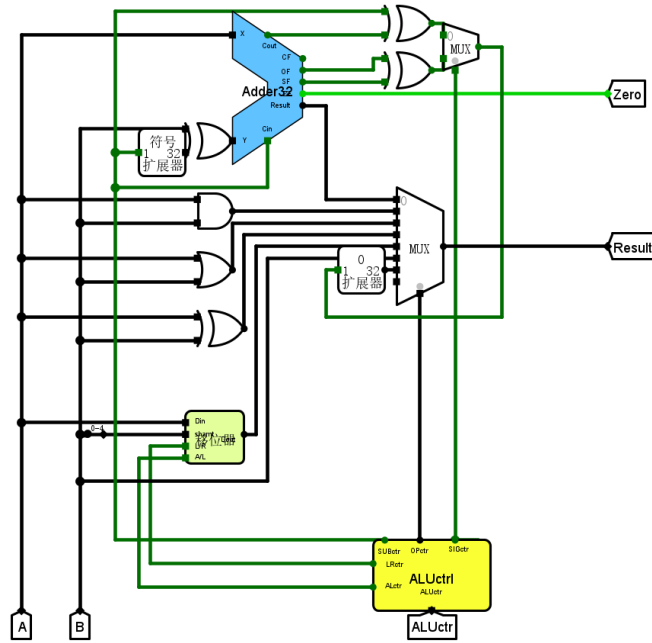


图 19: RV32I 算术逻辑部件电路图

5.3 仿真测试

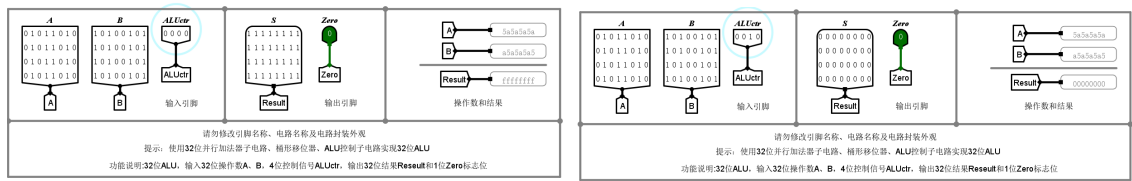


图 20: ALU 功能测试图

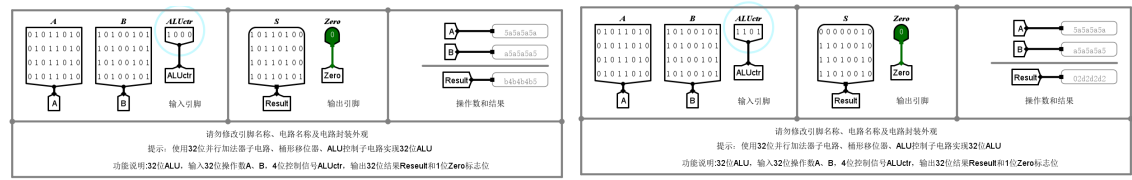


图 21: ALU 功能测试图（测试 3/测试 4）

表 1: ALU 功能测试表

序号	A	B	ALUctr<3:0>	Result	Zero
1	0x80000000	0x7fffffff	0 0 0 0	0xffffffff	0
2	0x01	0x7fffffff	0 0 0 1	0x80000000	0
3	0x80000000	0x7fffffff	0 0 1 0	0x00000001	0
4	0x80000000	0x7fffffff	0 0 1 1	0x00000000	0
5	0x80000000	0x7fffffff	0 1 0 0	0xffffffff	0
6	0x80000000	0x7fffffff	0 1 0 1	0x00000001	0
7	0x80000000	0x7fffffff	0 1 1 0	0xffffffff	0
8	0x80000000	0x7fffffff	0 1 1 1	0x00000000	0
9	0x80000000	0x7fffffff	1 0 0 0	0x00000000	0
10	0x80000000	0x7fffffff	1 1 0 1	0xffffffff	0
11	0x80000000	0x7fffffff	1 1 1 1	0x7fffffff	0
12	0x80000000	0x80000000	0 0 0 0	0x00000000	1
13	0x80000000	0x80000000	0 0 1 0	0x00000000	1
14	0x80000000	0x80000000	0 0 1 1	0x00000000	1
15	0x80000000	0x80000000	0 1 0 0	0x00000000	1
16	0x80000000	0x80000000	0 1 0 1	0x80000000	1
17	0x80000000	0x80000000	0 1 1 0	0x80000000	1
18	0x80000000	0x80000000	0 1 1 1	0x80000000	1
19	0x80000000	0x80000000	1 0 0 0	0x00000000	1
20	0x80000000	0x80000000	1 1 0 1	0x80000000	1
21	0x80000000	0x80000000	1 1 1 1	0x80000000	1
22	0x5a5a5a5a	0xa5a5a5a5	0 0 0 0	0xffffffff	0
23	0x5a5a5a5a	0xa5a5a5a5	0 0 0 1	0x4b4b4b40	0
24	0x5a5a5a5a	0xa5a5a5a5	0 0 1 0	0x00000000	0
25	0x5a5a5a5a	0xa5a5a5a5	0 0 1 1	0x00000000	0

表 2: ALU 功能测试表 (续)

序号	A	B	ALUctr<3:0>	Result	Zero
26	0x5a5a5a5a	0xa5a5a5a5	0 1 0 0	0xffffffff	0
27	0x5a5a5a5a	0xa5a5a5a5	0 1 0 1	0x02d2d2d2	0
28	0x5a5a5a5a	0xa5a5a5a5	0 1 1 0	0xffffffff	0
29	0x5a5a5a5a	0xa5a5a5a5	0 1 1 1	0x00000000	0
30	0x5a5a5a5a	0xa5a5a5a5	1 0 0 0	0xa4a4a4a5	0
31	0x5a5a5a5a	0xa5a5a5a5	1 1 0 1	0x02d2d2d2	0
32	0x5a5a5a5a	0xa5a5a5a5	1 1 1 1	0xa5a5a5a5	0

5.4 错误分析

本次实验没有遇到问题和错误。

6 思考题

6.1 64 位快速加法器

用 4 个 16 位先行进位加法器与一个 CLU 部件组成一个 64 位快速加法器。

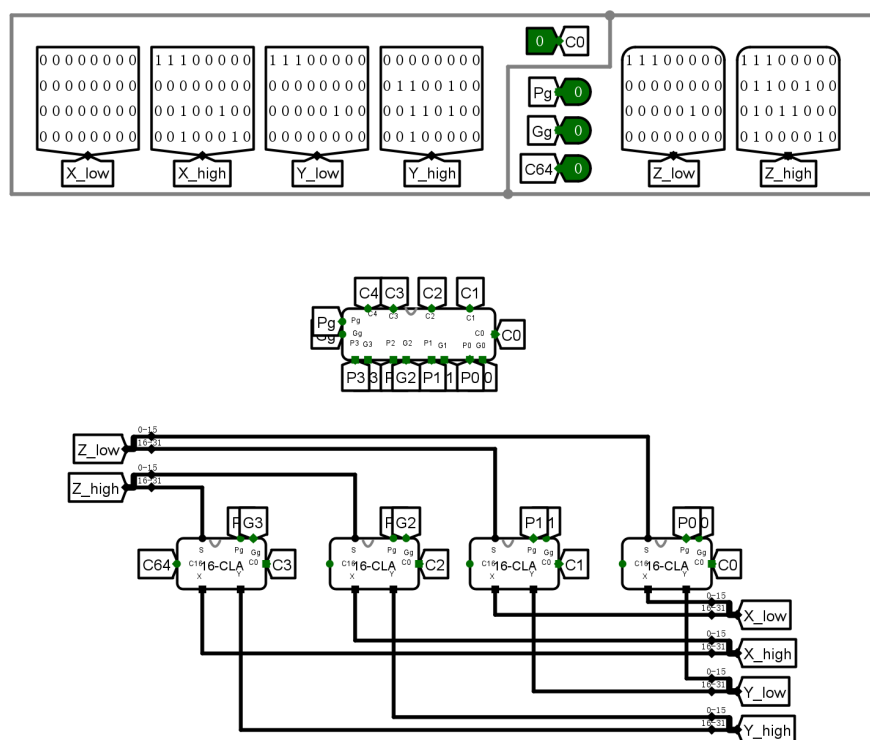


图 22: 64 位快速加法器

6.2 32 位无符号乘法器

设计一个 32 位无符号乘法器，输入为两个 32 位无符号数，输出为 64 位乘积。

- 如何在乘法器开始前进行清零
- 如何确保乘法器运算过程中 ALU 控制码不发生变化
- 如何确保乘法器运算过程中不输出中间值

6.3.1 关于清零

通过 D 触发器和时钟，在乘法控制信号输出 (增加一个 ALUctr 输出端口 $Mul_start = m_9 + m_{10}$) 时产生脉冲驱动 RST 位。

6.3.2 关于控制码不变

首先需要在乘法器电路中，增加一个 Done 输出端口，其值在计数器达到 33 时始终为 1。在 ALUctr 输入前加一个寄存器，通过改变其使能端来锁存控制码。具体地， $en = \overline{Mul_start} + Done$ 。当乘法开始时， Mul_start 为 1， en 为 0，控制码被锁存；当乘法结束时， $Done$ 为 1， en 为 1，控制码可以更新。

6.3.3 关于中间值输出

类似地，在 Result 输出前增加一个寄存器，采用一样的使能端控制逻辑。

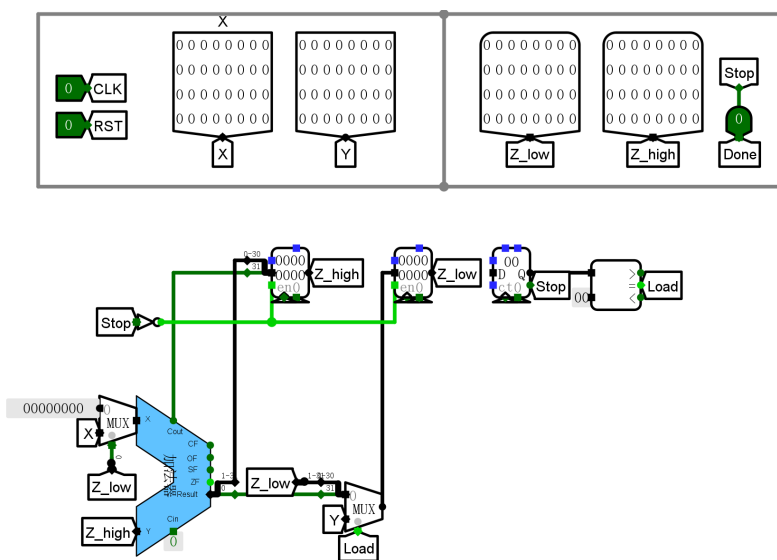


图 24: 修改后的乘法器设计电路

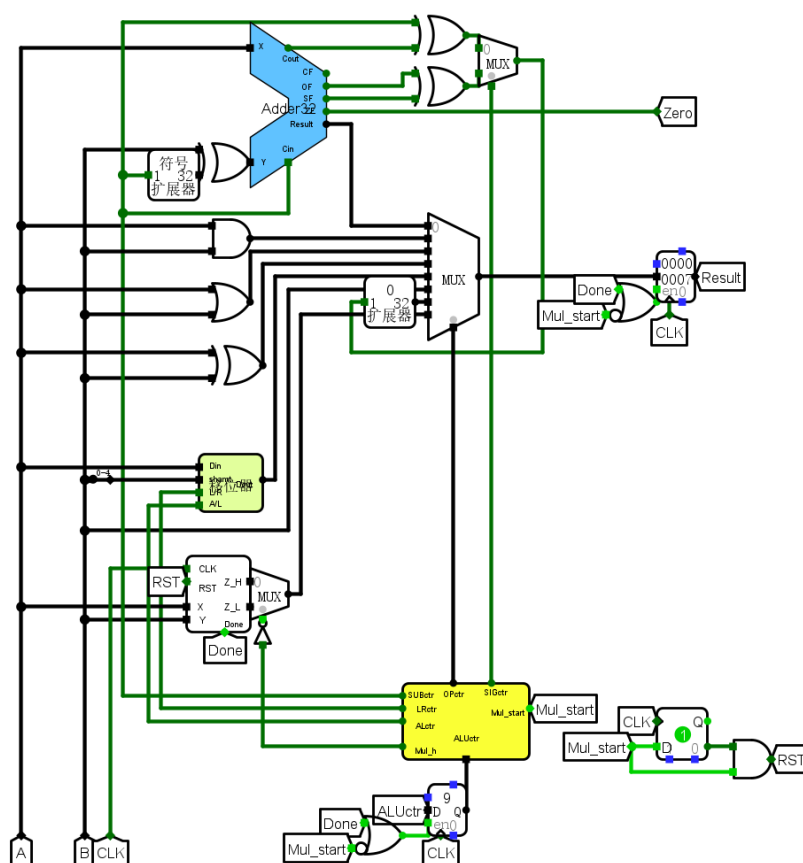
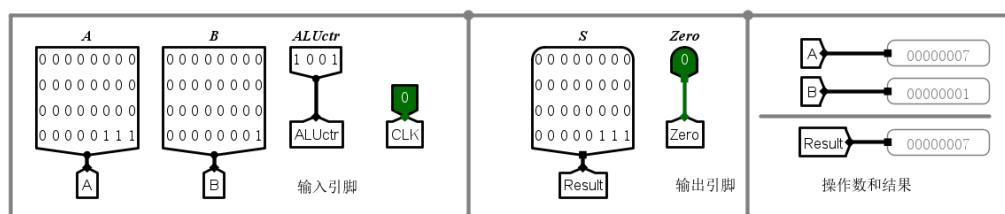


图 25: 修改后的 ALU 设计电路

7 验收表

序号	ALUctr	A	B	Result	Zero	说明
1	0000(ADD)	0x00000000	0x00000000	0x00000000	1	加法
2	0000(ADD)	0x00000001	0x00000001	0x00000002	0	加法
3	0000(ADD)	0x7FFFFFFF	0x00000001	0x80000000	0	加法
4	0000(ADD)	0xFFFFFFFF	0x00000001	0x00000000	1	加法
5	0000(ADD)	0x80000000	0x80000000	0x00000000	1	加法
6	0000(ADD)	0x12345678	0x9ABCDEF0	0xACF13567	0	加法
7	1000(SUB)	0x00000005	0x00000003	0x00000002	0	减法
8	1000(SUB)	0x00000003	0x00000005	0xFFFFFFFF	0	减法
9	1000(SUB)	0x80000000	0x00000001	0x7FFFFFFF	0	减法
10	1000(SUB)	0x00000000	0x00000001	0xFFFFFFFF	0	减法
11	1000(SUB)	0x12345678	0x9ABCDEF0	0x77777788	0	减法
12	0111(AND)	0xF0F0F0F0	0x0F0F0F0F	0x00000000	0	与运算
13	0111(AND)	0xFFFFFFFF	0x12345678	0xFFFFFFFF	0	与运算
14	0111(AND)	0x12345678	0x9ABCDEF0	0x12345670	0	与运算
15	0110(OR)	0xF0F0F0F0	0x0F0F0F0F	0xFFFFFFFF	0	或运算
16	0110(OR)	0x00000000	0x87654321	0x87654321	0	或运算
17	0110(OR)	0x12345678	0x9ABCDEF0	0x9ABCDEF8	0	或运算
18	0100(XOR)	0xAAAAAAAA	0x55555555	0xFFFFFFFF	0	异或运算
19	0100(XOR)	0xFFFFFFFF	0x12345678	0xEDCBA987	0	异或运算
20	0100(XOR)	0x12345678	0x9ABCDEF0	0x88888888	0	异或运算

表 3: 验收表 (续)

序号	ALUctr	A	B	Result	Zero	说明
21	0001(SLL)	0x00000001	0x00000000	0x00000001	0	逻辑左移
22	0001(SLL)	0x00000001	0x0000001F	0x80000000	0	逻辑左移
23	0001(SLL)	0x80000001	0x00000004	0x00000010	0	逻辑左移
24	0101(SRL)	0x80000000	0x0000001F	0x00000001	0	逻辑右移
25	0101(SRL)	0xFFFFFFFF	0x00000004	0x0FFFFFFF	0	逻辑右移
26	1101(SRA)	0x80000000	0x0000001F	0xFFFFFFFF	0	算数右移
27	1101(SRA)	0xFFFFFFFF	0x00000004	0xFFFFFFFF	0	算数右移
28	1101(SRA)	0x7FFFFFFF	0x00000004	0x07FFFFFF	0	算数右移
29	0010(SLT)	0xFFFFFFFF	0x00000001	0x00000001	0	有符号比较
30	0010(SLT)	0x7FFFFFFF	0x80000000	0x00000000	0	有符号比较
31	0010(SLT)	0x80000000	0x80000000	0x00000000	1	有符号比较
32	0011(SLTU)	0xFFFFFFFF	0x00000001	0x00000000	0	无符号比较
33	0011(SLTU)	0x00000001	0x00000001	0x00000000	1	无符号比较
34	0011(SLTU)	0x80000000	0x7FFFFFFF	0x00000000	0	无符号比较